

JDBC Update and Delete Operations

Update Operation in JDBC

Step 1: Create a Java Class

- Create a Java class with the `main()` method.
 - Handle exceptions using:
 - `throws Exception`, or
 - `try-catch` block.
-

Step 2: Load the JDBC Driver

Load the MySQL JDBC Driver.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 3: Add MySQL Connector JAR (Only for a New Project)

If you are creating a new project:

- Right-click the project.
- Select **Build Path**.
- Click **Configure Build Path**.
- Choose **Add External JARs**.
- Select the MySQL Connector JAR file.

Note: If the JAR is already added, you do not need to add it again.

Step 4: Create a Database Connection

Connect your Java program to the MySQL database.

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/jdbc_db",  
    "root",  
    "password"  
);
```

Replace:

- `jdbc_db` → Your database name
 - `root` → Your username
 - `password` → Your MySQL password
-

Step 5: Write the SQL Update Query

```
UPDATE register  
SET city = ?  
WHERE email = ?;
```

Remember

- Use `?` for parameter values.
 - Always use the `WHERE` clause.
 - Without `WHERE`, **all records will be updated.**
-

Step 6: Create a PreparedStatement

```
PreparedStatement ps = con.prepareStatement(query);
```

Step 7: Set Parameter Values

```
ps.setString(1, "Pune");  
ps.setString(2, "kamal@gmail.com");
```

Parameter	Value
1	New City
2	Email

Step 8: Execute the Query

```
int i = ps.executeUpdate();
```

`executeUpdate()` returns the number of affected rows.

Step 9: Check the Result

```
if(i > 0)  
{  
    System.out.println("Update Successful");  
}  
else  
{  
    System.out.println("Update Failed");  
}
```

Step 10: Close Resources

```
ps.close();  
con.close();
```

Delete Operation in JDBC

Step 1: Load Driver and Create Connection

These steps are the same as the Update operation.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/jdbc_db",  
    "root",  
    "password"  
);
```

Note: If you are using the same project, you do **not** need to add the MySQL Connector JAR again.

Step 2: Write the SQL Delete Query

```
DELETE FROM register  
WHERE email = ?;
```

Remember

- Always use the **WHERE** clause.
 - Without **WHERE**, all records will be deleted.
-

Step 3: Create a PreparedStatement

```
PreparedStatement ps = con.prepareStatement(query);
```

Step 4: Set Parameter Value

```
ps.setString(1, "kamal@gmail.com");
```

Step 5: Execute the Query

```
int i = ps.executeUpdate();
```

Step 6: Check the Result

```
if(i > 0)
{
    System.out.println("Deletion Successful");
}
else
{
    System.out.println("Deletion Failed");
}
```

Step 7: Close Resources

```
ps.close();
con.close();
```

Important Points

- ☒ Load the JDBC Driver before connecting.
 - ☒ Add the MySQL Connector JAR only once per project.
 - ☒ Use **PreparedStatement** instead of **Statement**.
 - ☒ Always use a **WHERE** clause in **UPDATE** and **DELETE**.
 - ☒ Set parameter indexes correctly (1, 2, 3, ...).
 - ☒ `executeUpdate()` returns the number of affected rows.
 - ☒ Close **PreparedStatement** and **Connection** after use.
-

Quick Summary

Operation	SQL Query	Method Used	Success Condition
INSERT	INSERT INTO table VALUES(...)	executeUpdate()	Rows > 0
UPDATE	UPDATE table SET column=? WHERE condition=?	executeUpdate()	Rows > 0
DELETE	DELETE FROM table WHERE condition=?	executeUpdate()	Rows > 0

Easy Memory Trick

Operation	Meaning
INSERT	Add a new record
UPDATE	Change an existing record
DELETE	Remove a record

Remember: All three operations use **PreparedStatement** and **executeUpdate()** in JDBC.

Video Link: [#3 Update and Delete Operation using JDBC || CRUD Operation || Java MyS...](#)